

Introduction to JavaScript

1. What is JavaScript?

- JavaScript is a **high-level, interpreted, lightweight, and dynamic scripting language**.
- It is primarily used to **create interactive and dynamic content** on websites (e.g., form validation, interactive maps, animations).
- JS is one of the core technologies of the **web**, alongside **HTML** and **CSS**.

2. Key Features of JavaScript

Feature	Description
Lightweight	Fast and optimized for minimal processing
Dynamic Typing	No need to declare variable types (var a = 10;; var b = "text";)
Interpreted	No compilation needed; interpreted by the browser
Event-Driven	Responds to user actions like clicks, hovers, keypresses
Object-Oriented	Supports object creation and manipulation
First-Class Functions	Functions can be assigned to variables, passed as arguments, etc.
Platform-Independent	Runs in all modern browsers regardless of the OS

3. History of JavaScript

Year	Event
1995	JavaScript created by Brendan Eich at Netscape
1996	Renamed from LiveScript to JavaScript
1997	Standardized as ECMAScript (ES) by ECMA International
2009	Node.js introduced for server-side JavaScript
2015	ES6 (ECMAScript 6) released with major updates

4. Where is JavaScript Used?

- Client-Side Web Development (main use)
- Server-Side Development (with Node.js)
- Game Development (e.g., Phaser)
- Mobile App Development (e.g., React Native)
- Desktop Applications (e.g., Electron)
- Automation & Scripting (e.g., Puppeteer)

5. Client-Side vs Server-Side JavaScript

Client-Side

Runs in browser

Used for UI interactions, DOM

Faster (no server request needed)

Server-Side

Runs on server (e.g., Node.js)

Used for handling databases, APIs

Slower due to network and I/O latency

6. Integration with HTML

Inline Script:

```
<button onclick="alert('Hello!')">Click Me</button>
```

Internal Script:

```
<script>  
  alert ("Hello from JavaScript!");  
</script>
```

External Script:

```
<script src="script.js"></script>
```

7. JavaScript vs Java

JavaScript

Interpreted, scripting language

Runs in browser

Used for web interactivity

No class-based structure required

Java

Compiled, object-oriented programming

Runs on JVM

Used for standalone or enterprise apps

Fully class-based

8. "Hello World" in JavaScript

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<script>
```

```
    alert ("Hello, World!");
```

```
    console.log ("Hello from console!");
```

```
    document. Write ("Hello on the page!");
```

```
</script>
```

```
</body>
```

```
</html>
```

9. Why Learn JavaScript?

- Used by **95%+ of websites**
- High demand in the job market
- Easy to learn and integrate with HTML/CSS
- Strong community and ecosystem (React, Angular, Vue, Node.js)
- Gateway to full-stack development

USBM

Detailed History of JavaScript

1. Creation (1995)

- **Who?**

JavaScript was created by **Brendan Eich**, a programmer at **Netscape Communications Corporation**.

- **Why?**

The goal was to create a **lightweight scripting language** that could:

- Run in the browser,
- Handle simple tasks like form validation and animation,
- Complement Java and HTML.

- **How long did it take?**

Brendan Eich developed the first version in **just 10 days** in May 1995.

- **Original name:**

It was first called **Mocha**, then briefly renamed to **LiveScript**.

- **Final name:**

In December 1995, it was renamed **JavaScript** as part of a marketing strategy to ride the popularity of **Java**, although Java and JavaScript are very different.

2. Standardization – ECMAScript (1996–1997)

- **Problem:**

As JavaScript's popularity grew, inconsistencies emerged in how it worked in different browsers.

- **Solution:**
JavaScript was submitted to the **ECMA International** standards organization.
- **Outcome:**
In **June 1997**, ECMA published the first standard for the language:
ECMAScript 1 (ES1)

3. Key ECMAScript Versions and Features

Version	Year	Key Features
ES1	1997	First version, basic syntax
ES2	1998	Minor edits
ES3	1999	Added try-catch, regular expressions, better string handling
ES4	Abandoned	Too complex, never released
ES5	2009	Strict mode ("use strict"), JSON support, array methods (forEach, map, filter)
ES6 (ES2015)	2015	Major upgrade: let, const, arrow functions, classes, template literals, promises, modules
ES7 (ES2016)	2016	Includes () method, exponentiation operator (**)
ES8 (ES2017)	2017	async/await, Object.entries(), Object.values()
ES9 (ES2018)	2018	Spread in objects, async iteration

Version	Year	Key Features
ES10 (ES2019)	2019	Flat (), flatMap (), trimStart(), trimEnd()
ES11 (ES2020)	2020	Optional chaining (?.), nullish coalescing (??), Promise.allSettled()
ES12+ (2021 onwards)	Every year since	Continuous updates like replaceAll(), top-level await, logical assignment operators

4. JavaScript in the Real World

- **2009: Node.js released**
JavaScript could now run **outside the browser**, allowing **server-side** development using JS.
- **2010–2015: Rise of Frameworks**
 - Libraries like **jQuery** made DOM manipulation easy.
 - Frameworks like **AngularJS**, **React**, and **Vue** transformed how we build dynamic web apps.
- **2015 onwards: JS Everywhere**
 - **React Native** → Mobile Apps
 - **Electron** → Desktop Apps
 - **Next.js**, **Express.js** → Full-stack apps

5. Why the Name “JavaScript”?

- **Marketing move by Netscape** to attract Java developers.
- Java and JavaScript are completely different:

- Java: compiled, class-based, statically typed
- JavaScript: interpreted, prototype-based, dynamically typed

6. Current Status (as of 2025)

- JavaScript is one of the **most-used programming languages** globally.
- It powers everything from **simple websites to large-scale web applications**, mobile apps, games, and IoT devices.

Summary Timeline:

Year	Event
1995	JavaScript created as Mocha at Netscape
1996	Renamed to LiveScript, then JavaScript
1997	ECMAScript 1 (ES1) standardized
2009	Node.js introduced
2015	ES6 brought modern features
2020+	JS supports full-stack, mobile, and modern apps

What is a Scripting Language?

A **scripting language** is a **programming language** that is used to **automate tasks** and **control other programs or environments**, often inside a larger system like a web browser or operating system.

Key Features of Scripting Languages:

Feature	Description
◆ Interpreted	Code runs line-by-line by an interpreter (no need for compilation).
◆ Lightweight	Designed for short tasks, fast development, and ease of use.
◆ High-level syntax	Easy to read and write, close to human language.
◆ Embedded	Often embedded inside HTML or applications (e.g., JS in web pages).
◆ Dynamic typing	No need to define variable types explicitly.
◆ Rapid development	Great for writing quick scripts and prototypes.

How is it Different from Programming Languages?

Programming Language	Scripting Language
Used to build full applications	Used to automate small tasks or control environments

Programming Language Scripting Language

Usually compiled

Usually interpreted

Examples: C, C++, Java

Examples: JavaScript, Python, PHP, Bash

Examples of Scripting Languages:

Language Common Use

JavaScript Browser scripting, interactivity

Python Automation, AI, web development

PHP Server-side web scripting

Bash Linux shell scripting

Perl Text processing and file automation

Ruby Web development, automation

JavaScript as a Scripting Language:

- Runs **inside web browsers** to control web page behavior.
- Can:
 - Respond to user events (click, type)
 - Modify HTML/CSS dynamically
 - Validate forms
 - Communicate with servers using AJAX/fetch

Simple Example of a Script:

```
<script>
```

```
  alert("Welcome to JavaScript!");
```

```
</script>
```

- This script shows an alert box when the page loads.
- No compiling needed — just embed and run!

USBM

Basics of JavaScript

1. Variables in JavaScript (var, let, const)

What is a Variable?

- A variable is a container used to store **data** or **values**.
- In JS, variables are declared using var, let, or const.

var:

- **Function-scoped**
- Can be **redeclared** and **reassigned**
- Gets **hoisted** (moved to the top) with undefined

```
var x = 10;
```

```
var x = 20; // allowed
```

```
x = 30; // allowed
```

let:

- **Block-scoped**
- Cannot be **redeclared** in the same scope
- Can be **reassigned**

```
let y = 10;
```

```
y = 20; // allowed
```

```
// let y = 30; ❌ Not allowed in the same scope
```

const:

- **Block-scoped**
- Cannot be **redeclared** or **reassigned**
- Must be **initialized** at declaration

```
const PI = 3.14;
```

```
// PI = 4; ❌ Not allowed
```

2. Data Types

2.1 Primitive Data Types (Immutable, stored by value):

1. Number – e.g. 10, 3.14
2. String – e.g. 'Hello', "World"
3. Boolean – true / false
4. undefined – variable declared but not assigned
5. null – intentionally empty value
6. Symbol – unique and immutable (ES6)
7. BigInt – large integers (e.g.,
1234567890123456789012345678901234567890n)

```
let a = 10;    // Number
```

```
let b = "Avijit"; // String
```

```
let c = true;   // Boolean
```

2.2 Reference Data Types (Mutable, stored by reference):

1. **Objects** – collections of key-value pairs
2. **Arrays** – ordered list of values
3. **Functions** – blocks of code
4. **Dates, RegExp**, etc.

```
let person = { name: "Avijit", age: 25 };
```

```
let arr = [1, 2, 3];
```

3. Operators in JavaScript

3.1 Arithmetic Operators:

Used for mathematical calculations.

Operator	Description	Example
+	Addition	a + b
-	Subtraction	a - b
*	Multiplication	a * b
/	Division	a / b
%	Modulus (remainder)	a % b
**	Exponentiation	2 ** 3 (=> 8)
++	Increment	a++
--	Decrement	a--

3.2 Assignment Operators:

Operator	Description	Example
=	Assign	a = 10
+=	Add and assign	a += 5 (a = a + 5)
-=	Subtract and assign	a -= 3
*=	Multiply and assign	a *= 2
/=	Divide and assign	a /= 4
%=	Modulo and assign	a %= 3

3.3 Comparison Operators:

Operator	Description	Example
==	Equal to (loose)	5 == '5' → true
===	Equal (strict)	5 === '5' → false
!=	Not equal	a != b
!==	Not equal (strict)	a !== b
>	Greater than	a > b
<	Less than	a < b
>=	Greater or equal	a >= b
<=	Less or equal	a <= b

3.4 Logical Operators:

Operator Description Example

&& Logical AND true && false → false

,

! Logical NOT !true → false

3.5 Bitwise Operators:

Used for bit-level operations.

Operator Description

& AND

,

^ XOR

~ NOT

<< Left shift

>> Right shift

3.6 Ternary Operator:

condition ? valueIfTrue : valueIfFalse;

Example:

```
let age = 20;
```

```
let result = (age >= 18) ? "Adult" : "Minor";
```

4. Type Conversion

Implicit Type Conversion (Type Coercion):

JS automatically converts data types when needed.

```
'5' + 1    // "51" (string)
```

```
'5' - 1    // 4 (number)
```

```
true + 1   // 2
```

Explicit Type Conversion:

Using functions to manually convert types:

```
Number("123"); // 123
```

```
String(100);   // "100"
```

```
Boolean(0);    // false
```

```
parseInt("10.5"); // 10
```

```
parseFloat("10.5");// 10.5
```

5. Comments in JavaScript

Single-line comment:

```
// This is a single-line comment
```

Multi-line comment:

```
/*
```

```
This is a
```

```
multi-line comment
```

*** /**

Comments are ignored during execution and are useful for documentation.

Summary Table

Concept	Summary
var, let, const	Different variable declaration keywords (scope & behavior differ)
Data Types	Primitive (value-based), Reference (object-based)
Operators	Used for performing operations (Arithmetic, Logical, etc.)
Type Conversion	Implicit (automatic) or Explicit (manual)
Comments	Used to explain code or disable code blocks